

UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Laboratório 3 – *Crossbar Switch* e Multiplexadores

Pedro Lucas Moreira

Relatório – Linguagens de Descrição de Hardware – 2026/2 – Outubro de 2026

Professor(a) Orientador(a): Tiago Almeida

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Pedro Lucas Moreira

Laboratório 3 – *Crossbar Switch* e Multiplexadores

Relatório apresentado como parte das atividades da disciplina Linguagens de Descrição de Hardware – 2026/2, Curso de Ciência da Computação, Universidade Federal do Tocantins.

PALMAS / TO

2026

1 QUESTÃO 1 – *CROSSBAR SWITCH*

O componente `xbar` possui duas entradas de dados (x_1, x_2), um bit de controle S e duas saídas (y_1, y_2). Com $S = 0$ os dados passam direto ($y_1 = x_1$, $y_2 = x_2$); com $S = 1$ eles se cruzam ($y_1 = x_2$, $y_2 = x_1$).

Para que o mesmo código possa ser gravado na placa DE1, as portas usam os nomes da pinagem definida no `compile.tcl`:

Sinal	Porta VHDL	Pino
x_1	`SWITCH_PINS(0)` (SW0)	PIN_L22
x_2	`SWITCH_PINS(1)` (SW1)	PIN_L21
S	`SWITCH_PINS(2)` (SW2)	PIN_M22
y_1	`LEDR0`	PIN_R20
y_2	`LEDR1`	PIN_R19

1.1 Item (a) – `WITH`, `SELECT` e `WHEN`

Cada saída é escolhida por uma atribuição selecionada controlada por S.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- ============================================================
5  -- ITEM (a): crossbar switch (xbar) com WITH / SELECT / WHEN.
6  --
7  -- SWITCH_PINS(0) = x1
8  -- SWITCH_PINS(1) = x2
9  -- SWITCH_PINS(2) = S  (bit de controle)
10 -- LEDR0          = y1
11 -- LEDR1          = y2
12 --
13 -- S = '0' -> passa direto : y1 = x1, y2 = x2
14 -- S = '1' -> cruza        : y1 = x2, y2 = x1
15 -- ============================================================
16 entity crossbar_switch is
17     port (
```

```
        SWITCH_PINS : in  std_logic_vector(2 downto 0);
        LEDR0       : out std_logic;
        LEDR1       : out std_logic
    );
end entity crossbar_switch;

architecture rtl of crossbar_switch is
begin
    with SWITCH_PINS(2) select
        LEDR0 <= SWITCH_PINS(0) when '0',
                 SWITCH_PINS(1) when others;

    with SWITCH_PINS(2) select
        LEDR1 <= SWITCH_PINS(1) when '0',
                 SWITCH_PINS(0) when others;

end architecture rtl;
```

1.2 Item (b) – WHEN ELSE

Mesmo circuito com atribuição condicional: uma linha por saída.

```
library ieee;
use ieee.std_logic_1164.all;

-- ============================================================
-- ITEM (b): mesmo crossbar switch, agora com WHEN / ELSE.
--
-- SWITCH_PINS(0) = x1
-- SWITCH_PINS(1) = x2
-- SWITCH_PINS(2) = S  (bit de controle)
-- LEDR0          = y1
-- LEDR1          = y2
--
-- S = '0' -> passa direto : y1 = x1, y2 = x2
-- S = '1' -> cruza        : y1 = x2, y2 = x1
-- ============================================================
entity crossbar_switch is
    port (
```

```
        SWITCH_PINS : in  std_logic_vector(2 downto 0);
        LEDR0       : out std_logic;
        LEDR1       : out std_logic
    );
end entity crossbar_switch;

architecture rtl of crossbar_switch is
begin
    LEDR0 <= SWITCH_PINS(0) when SWITCH_PINS(2) = '0' else SWITCH_PINS
        (1);
    LEDR1 <= SWITCH_PINS(1) when SWITCH_PINS(2) = '0' else SWITCH_PINS
        (0);

end architecture rtl;
```

1.3 Item (c) – PROCESS

Mesmo circuito dentro de um processo. Como todas as entradas estão na lista de sensibilidade e as duas saídas recebem valor nos dois ramos do `if`, o circuito continua puramente combinacional.

```
library ieee;
use ieee.std_logic_1164.all;

-- ============================================================
-- ITEM (c): mesmo crossbar switch, agora com PROCESS.
--
-- SWITCH_PINS(0) = x1
-- SWITCH_PINS(1) = x2
-- SWITCH_PINS(2) = S  (bit de controle)
-- LEDR0          = y1
-- LEDR1          = y2
--
-- S = '0' -> passa direto : y1 = x1, y2 = x2
-- S = '1' -> cruza        : y1 = x2, y2 = x1
-- ============================================================
entity crossbar_switch is
    port (
        SWITCH_PINS : in  std_logic_vector(2 downto 0);
```

```
19          LEDR0       : out std_logic;
20          LEDR1       : out std_logic
21      );
22  end entity crossbar_switch;
23
24  architecture rtl of crossbar_switch is
25  begin
26      -- Lista de sensibilidade com todas as entradas: circuito
            combinacional.
27      process (SWITCH_PINS)
28      begin
29          if SWITCH_PINS(2) = '0' then
30              LEDR0 <= SWITCH_PINS(0);
31              LEDR1 <= SWITCH_PINS(1);
32          else
33              LEDR0 <= SWITCH_PINS(1);
34              LEDR1 <= SWITCH_PINS(0);
35          end if;
36      end process;
37
38  end architecture rtl;
```

1.4 Simulação dos itens (a), (b) e (c)

As três versões têm a mesma *entity* (`crossbar_switch`), então um único testbench serve para todas; os scripts `a.do`, `b.do` e `c.do` mudam apenas o arquivo compilado. O testbench percorre as 8 combinações de entrada, com 10 ns entre cada uma.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- Testbench dos itens (a), (b) e (c).
5  -- As tres versoes tem a mesma entity (crossbar_switch), entao o
6  -- mesmo testbench serve para todas. Basta compilar a versao
7  -- desejada antes (ver a.do, b.do, c.do).
8  entity tb_crossbar_switch is
9  end entity tb_crossbar_switch;
10
11 architecture sim of tb_crossbar_switch is
```

```
12      signal sw          : std_logic_vector(2 downto 0); -- S & x2 & x1
13      signal y1, y2      : std_logic;
14  begin
15      uut: entity work.crossbar_switch
16          port map (
17              SWITCH_PINS => sw,
18              LEDR0       => y1,
19              LEDR1       => y2
20          );
21
22      stim: process
23      begin
24          --       S x2 x1
25          sw <= "000"; wait for 10 ns; -- direto: y1=0 y2=0
26          sw <= "001"; wait for 10 ns; -- direto: y1=1 y2=0
27          sw <= "010"; wait for 10 ns; -- direto: y1=0 y2=1
28          sw <= "011"; wait for 10 ns; -- direto: y1=1 y2=1
29          sw <= "100"; wait for 10 ns; -- cruza : y1=0 y2=0
30          sw <= "101"; wait for 10 ns; -- cruza : y1=0 y2=1
31          sw <= "110"; wait for 10 ns; -- cruza : y1=1 y2=0
32          sw <= "111"; wait for 10 ns; -- cruza : y1=1 y2=1
33          wait;
34      end process;
35
36  end architecture sim;
```

As três simulações (ModelSim, arquivos `a.vcd`, `b.vcd` e `c.vcd`) produziram exatamente o mesmo resultado:

S	x_2	x_1	y_1	y_2	Modo
0	0	0	0	0	direto
0	0	1	1	0	direto
0	1	0	0	1	direto
0	1	1	1	1	direto
1	0	0	0	0	cruzado
1	0	1	0	1	cruzado
1	1	0	1	0	cruzado
1	1	1	1	1	cruzado

1.5 Item (d) – Cascata com `GENERIC` e `GENERATE`

A *entity* `xbar_chain` recebe o número de estágios pelo `generic N`. Os vetores `top` e `bot` representam as linhas de cima e de baixo entre os estágios: a entrada do primeiro estágio é VCC (`'1'`) em cima e GND (`'0'`) embaixo, e o `for generate` instancia N cópias do `crossbar_switch` do item (a), cada uma controlada por uma chave S_i. A saída y_1 do último estágio vai para o LED.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- ============================================================
5  -- ITEM (d): N xbars em cascata (GENERIC + GENERATE).
6  --
7  -- Entrada do 1o estagio: x1 = VCC ('1'), x2 = GND ('0').
8  -- Estagio i usa SWITCH_PINS(i) como bit de controle S_i.
9  -- Saida y1 do ultimo estagio vai para o LED (LEDR0).
10 --
11 -- Cada chave ligada troca as linhas de lugar. Logo o LED acende
12 -- quando o numero de chaves ligadas e PAR.
13 --
14 -- Usa o crossbar_switch do item (a): a.vhdl.
15 -- ============================================================
16 entity xbar_chain is
17     generic (
18         N : integer := 5
19     );
```

```
20      port (
21          SWITCH_PINS : in  std_logic_vector(N-1 downto 0);
22          LEDR0       : out std_logic
23      );
24  end entity xbar_chain;
25
26  architecture structural of xbar_chain is
27      -- top(i) / bot(i) = linhas de cima (y1) e de baixo (y2) na entrada
            do estagio i
28      signal top : std_logic_vector(N downto 0);
29      signal bot : std_logic_vector(N downto 0);
30  begin
31      top(0) <= '1'; -- VCC
32      bot(0) <= '0'; -- GND
33
34      xbar_gen: for i in 0 to N-1 generate
35          xbar_inst: entity work.crossbar_switch
36              port map (
37                  SWITCH_PINS(0) => top(i),          -- x1
38                  SWITCH_PINS(1) => bot(i),          -- x2
39                  SWITCH_PINS(2) => SWITCH_PINS(i),  -- S_i
40                  LEDR0          => top(i + 1),      -- y1
41                  LEDR1          => bot(i + 1)       -- y2
42              );
43      end generate xbar_gen;
44
45      LEDR0 <= top(N);
46
47  end architecture structural;
```

Cada chave ligada troca as linhas de lugar. Assim, o valor de VCC volta para a linha de cima sempre que um número **par** de chaves estiver ligado, ou seja, LED $= \overline{S_0 \oplus S_1 \oplus \cdots \oplus S_{N-1}}$. Por isso, mudar qualquer chave sozinha inverte o estado do LED, que é o comportamento do interruptor de corredor citado no enunciado.

O testbench (`tb_xbar_chain.vhdl`, com $N = 5$) confirmou esse comportamento:

$S_4 \ldots S_0$	Chaves ligadas	LED
00000	0	1
00001	1	0
00011	2	1
10101	3	0
11110	4	1
11111	5	0

1.6 Item (e) – `demo_setup` com 5 estágios

O `demo_setup` instancia o `xbar_chain` com `N => 5`, ligando SW0 a SW4 nas chaves S_0 a S_4 e a saída no LEDR0. O projeto (`compile-2.tcl`, projeto `demo_setup_5`) compilou sem erros e ocupou 2 elementos lógicos: o sintetizador reduz toda a cascata de multiplexadores ao XNOR das 5 chaves, como mostra o *netlist* no RTL Viewer.

```
library ieee;
use ieee.std_logic_1164.all;

-- ============================================================
-- ITEM (e): demo_setup com 5 estagios.
-- SW0..SW4 = S0..S4, LEDR0 = saida.
-- ============================================================
entity demo_setup is
    port (
        SWITCH_PINS : in  std_logic_vector(4 downto 0);
        LEDR0       : out std_logic
    );
end entity demo_setup;

architecture rtl of demo_setup is
begin
    chain_inst: entity work.xbar_chain
        generic map (N => 5)
        port map (
            SWITCH_PINS => SWITCH_PINS,
            LEDR0       => LEDR0
        );
```

```
23
24 end architecture rtl;
```

1.7 Item (f) – `demo_setup` com 8 estágios

Igual ao item (e), alterando apenas `N => 8` e o tamanho do vetor de chaves (SW0 a SW7). O projeto (`compile-3.tcl`, projeto `demo_setup_8`) compilou sem erros e ocupou 3 elementos lógicos.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- ============================================================
5  -- ITEM (f): demo_setup com 8 estagios.
6  -- Igual ao item (e), so muda o N (e o tamanho do vetor).
7  -- SW0..SW7 = S0..S7, LEDR0 = saida.
8  -- ============================================================
9  entity demo_setup is
10     port (
11         SWITCH_PINS : in  std_logic_vector(7 downto 0);
12         LEDR0       : out std_logic
13     );
14 end entity demo_setup;
15
16 architecture rtl of demo_setup is
17 begin
18     chain_inst: entity work.xbar_chain
19         generic map (N => 8)
20         port map (
21             SWITCH_PINS => SWITCH_PINS,
22             LEDR0       => LEDR0
23         );
24
25 end architecture rtl;
```

2 QUESTÃO 2 – MULTIPLEXADOR COM DECODIFICADOR

2.1 Item (a) – Decodificador 2 para 4

O decodificador recebe a seleção w_1w_0 e o sinal de habilitação En. Com $En = 0$ todas as saídas ficam em 0; com $En = 1$ apenas a saída y_i correspondente ao valor de w_1w_0 vai para 1. A implementação usa uma única atribuição selecionada sobre o vetor `En & w1 & w0`, sem processo.

Sinal	Porta VHDL	Pino
w_0	`SWITCH_PINS(0)` (SW0)	PIN_L22
w_1	`SWITCH_PINS(1)` (SW1)	PIN_L21
En	`SWITCH_PINS(2)` (SW2)	PIN_M22
$y_0 \ldots y_3$	`LEDR0 ... LEDR3`	PIN_R20, R19, U19, Y19

```vhdl
library ieee;
use ieee.std_logic_1164.all;

-- ============================================================
-- QUESTAO 2, ITEM (a): decodificador 2 para 4 com enable
-- (WITH / SELECT, sem processo).
--
-- SWITCH_PINS(0) = w0
-- SWITCH_PINS(1) = w1
-- SWITCH_PINS(2) = En
-- LEDR0..LEDR3   = y0..y3
--
-- En = '0' -> todas as saidas em '0'
-- En = '1' -> so a saida y(w1 w0) vai para '1'
-- ============================================================
entity dec2_4 is
    port (
        SWITCH_PINS : in  std_logic_vector(2 downto 0);
        LEDR0       : out std_logic;
        LEDR1       : out std_logic;
        LEDR2       : out std_logic;
```

```
22          LEDR3       : out std_logic
23      );
24  end entity dec2_4;
25
26  architecture rtl of dec2_4 is
27      signal y : std_logic_vector(3 downto 0); -- y3 y2 y1 y0
28  begin
29      --                      En w1 w0
30      with SWITCH_PINS select
31          y <= "0001" when "100",
32               "0010" when "101",
33               "0100" when "110",
34               "1000" when "111",
35               "0000" when others; -- En = '0'
36
37      LEDR0 <= y(0);
38      LEDR1 <= y(1);
39      LEDR2 <= y(2);
40      LEDR3 <= y(3);
41
42  end architecture rtl;
```

O testbench (`tb_dec2_4.vhdl`) percorre as 8 combinações de entrada:

En	w_1	w_0	y_3	y_2	y_1	y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

2.2 Item (b) – Lógica extra

O circuito da caixa pontilhada da Figura 1 faz o AND de cada entrada de dados w_i com a saída y_i do decodificador e junta os quatro resultados em um OR:

$$f = w_0 y_0 + w_1 y_1 + w_2 y_2 + w_3 y_3$$

Como o decodificador ativa apenas um y_i por vez, f é igual à entrada w_i do canal selecionado. Na placa, SW0 a SW3 são w_0 a w_3, SW4 a SW7 são y_0 a y_3 e o LEDR0 mostra f.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

-- ============================================================
-- QUESTAO 2, ITEM (b): logica extra (caixa pontilhada da Fig. 1),
-- sem processo. Cada entrada w_i passa por um AND com a saida y_i
-- do decodificador e o OR dos quatro resultados gera f.
--
-- SWITCH_PINS(3 downto 0) = w0..w3
-- SWITCH_PINS(7 downto 4) = y0..y3
-- LEDR0                   = f
-- ============================================================
entity extra_logic is
    port (
        SWITCH_PINS : in  std_logic_vector(7 downto 0);
        LEDR0       : out std_logic
    );
end entity extra_logic;

architecture rtl of extra_logic is
    signal w : std_logic_vector(3 downto 0);
    signal y : std_logic_vector(3 downto 0);
begin
    w <= SWITCH_PINS(3 downto 0);
    y <= SWITCH_PINS(7 downto 4);

    LEDR0 <= (w(0) and y(0)) or
             (w(1) and y(1)) or
             (w(2) and y(2)) or
             (w(3) and y(3));

end architecture rtl;
```

O testbench (`tb_extra_logic.vhdl`) ativa um canal de cada vez, testando o w_i do canal em 1 e em 0 enquanto as outras entradas ficam com o valor oposto, para garantir

que só o canal ativo influencia a saída:

$y_3 \dots y_0$	$w_3 \dots w_0$	f
0000	1111	0
0001	0001	1
0001	1110	0
0010	0010	1
0010	1101	0
0100	0100	1
0100	1011	0
1000	1000	1
1000	0111	0

Os dois itens foram compilados no Quartus (`compile.tcl` e `compile-2.tcl`) sem erros.